

Estruturas de Dados

Tipos Abstratos de Dados (TAD) e Lista

Curso: Tecnologia em Análise e Desenvolvimento de Sistemas

Disciplina: Estruturas de Dados

Foco: compreender o conceito de TAD e implementar uma lista estática em C

1. Tipos de Dados

Um tipo de dados caracteriza o conjunto de valores que uma variável pode assumir.

Também define operações que podem ser realizadas sobre esses valores.

Exemplos de tipos elementares: inteiro, real, caractere e string.

Exemplo: inteiros e reais suportam operações aritméticas; strings suportam concatenação e segmentação.

2. Tipo Abstrato de Dados (TAD)

Um TAD especifica um conjunto de dados e as operações que podem ser executadas sobre esses dados.

Os dados podem ser frequentemente heterogêneos.

O foco está na entidade e nas operações disponíveis, e não apenas na representação interna.

Um mesmo TAD pode ter diferentes formas de implementação.

Exemplo de TAD: Estudante

Dados: nome, idade e matrícula.

Operações: duplicar informação, validar matrícula e verificar idade.

Na prática, o material apresenta a implementação com uma struct e funções que operam sobre ela.

```
struct Estudante {  
    char nome[50];  
    int idade;  
    int matrícula;  
};  
  
int maiorDeIdade(struct Estudante e);  
int matriculaValida(struct Estudante e);
```

TAD × Implementação

TAD → especificação dos dados e operações.

Implementação → estrutura concreta e funções usadas para realizar as operações.

A separação favorece a organização do programa e permite diferentes implementações para um mesmo TAD.

3. Lista como TAD

Lista é um TAD comum para armazenar informações.

Exemplo cotidiano: uma relação de itens escrita em uma folha.

Operações usuais:

- Criação da lista
- Inclusão de elemento
- Exclusão de elemento
- Contagem e exibição dos elementos

Lista com implementação estática

Uma forma de implementar a lista é usar um **vetor de strings**.

No exemplo do material, cada string representa um item da lista.

O espaço é definido na criação da variável e tem capacidade máxima fixa.

Exemplo: lista de compras.

```
#define MaxItens 10  
#define TamItem 20  
typedef char tpLista[MaxItens][TamItem];  
  
tpLista L1;
```


Operação: criar lista vazia

Para iniciar uma lista, é preciso criar uma lista vazia.

A função `criaLista` zera os strings da lista.

Um string vazio é identificado por `l[i][0] = '\0'`.

```
void criaLista(tpLista l)
{
    int i;
    for (i=0; i<MaxItens; i++)
        l[i][0] = '\0';
}
```

Operação: imprimir a lista

Percorre os elementos a partir da primeira posição.

A impressão termina quando encontra um elemento vazio ou chega ao limite.

A função usa `strlen` para verificar se há conteúdo na posição.

```
void imprimeLista(tpLista l)
{
    int i;
    printf("\nItens da lista\n");
    for(i=0; i<MaxItens && strlen(l[i])>0; i++)
        printf("\n%s", l[i]);
}
```

Operação: contar elementos

A lógica é semelhante à impressão, mas em vez de exibir os itens, conta as posições ocupadas.

A função para ao encontrar o primeiro string vazio ou ao atingir MaxItens.

O índice encontrado equivale ao número de elementos.

```
int contaLista(tpLista l)
{
    int i;
    for (i=0; i<MaxItens && strlen(l[i])>0; i++);
    return(i);
}
```

Operação: inserir elemento

Percorre a lista até encontrar a primeira posição vazia.

Se houver espaço, copia o item para essa posição com strcpy.

Se todas as posições estiverem ocupadas, informa que a lista está cheia.

```
void insereLista(tpLista l, char *item)
{
    int i;

    for(i=0; i<MaxItens && strlen(l[i])>0; i++);

    if (i<MaxItens)
        strcpy(l[i], item);
    else
        printf("Lista cheia, não pode inserir");
}
```

Operação: retirar elemento

Primeiro, a função procura o item a ser removido.

Ao encontrar o item, os elementos seguintes são deslocados uma posição para trás.

O objetivo é manter os elementos sem espaços entre eles.

Se o item não for encontrado, a função informa essa situação.

Retirada: ideia do deslocamento

Antes:

```
[ carne | arroz | alface | feijão | ... ]
```

Remover "arroz":

```
[ carne | alface | feijão | ... ]
```

Passo essencial:

cada elemento posterior ocupa a posição anterior.

Exemplo de código da retirada

- O deslocamento elimina o “buraco” deixado pelo elemento removido.
- Ao final, a última posição é zerada.
- A busca considera três possibilidades: fim do espaço, elemento vazio ou item encontrado.

```
void retiraLista(tpLista l, char *item)
{
    int i;

    for(i=0; i<MaxItens && strlen(l[i])>0 &&
        (strcmp(l[i],item) != 0); i++);

    if (i<MaxItens && (strcmp(l[i],item) == 0))
    { // Encontrou o elemento buscado
        printf("\nItem (%s) encontrado, removendo", item);
        if (i < (MaxItens-1))
        { // Tem elementos depois dele, puxa para cima
            for (;i<(MaxItens-1) && strlen(l[i])>0;i++)
                strcpy(l[i],l[i+1]);
        }
        // Zera o último elemento do espaço da lista
        l[MaxItens-1][0] = '\0';
    }
    else // Não encontrou o elemento buscado
        printf("\nItem (%s) não encontrado",item);
}
```

Integração das operações

criaLista → prepara a lista vazia

insereLista → adiciona um item na primeira posição livre

contaLista → informa quantos elementos estão ocupados

imprimeLista → exibe os itens

retiraLista → localiza, remove e reorganiza os elementos

O programa principal ainda precisa oferecer uma interface ao usuário e chamar a função correspondente.

Atividade

Implemente um programa principal para operar a lista.

Crie um menu com as opções: criar, inserir, retirar, contar e imprimir.

Teste a inserção até atingir a capacidade máxima.

Teste a retirada do primeiro, de um elemento intermediário e do último.

Teste também a tentativa de retirar um item inexistente.

Discuta: quais são as vantagens e limitações de uma lista com implementação estática?